

n8n-io/n8n

n8n : ingénierie solide, cadence industrielle, une seule vraie contrainte — la licence. Adopter en auto-hébergé, ne pas forker.

Fair-code workflow automation platform with native AI capabilities. Combine visual building with custom code, self-host or cloud, 400+ integrations.

16

BUS FACTOR
sur 12 semaines

500

COMMITTS
sur 12 semaines

28

DÉPENDANCES
0 en production

Sustainable
Use

LICENCE

Que fait ce projet et quelle est son architecture ?

À retenir : ce n'est pas un projet communautaire qui a grossi, c'est un monorepo d'éditeur — 53 entrées à la racine dont la moitié sont des garde-fous d'ingénierie.

<code>.agents/</code>	règles de revue pour agents IA
<code>.claude/</code>	config Claude Code
<code>.devcontainer/</code>	environnement de dev conteneurisé
<code>.githubhooks/</code>	hooks git
<code>.github/</code>	workflows CI, actions, scripts
<code>.opencode/</code>	config agent OpenCode
<code>.vscode/</code>	réglages VS Code
<code>assets/</code>	images et ressources
<code>docker/</code>	images Docker
<code>docs/</code>	documentation et schémas générés
<code>packages/</code>	code source, monorepo pnpm
<code>patches/</code>	patches de dépendances pnpm
<code>scripts/</code>	scripts de build et de dev
<code>security/</code>	outillage sécurité
<code>.actrc</code>	config act, Actions en local
<code>.aikido</code>	config scanner sécurité Aikido

... et 37 autres entrées à la racine du dépôt

Plateforme d'automatisation de workflows et d'agents IA, visuelle avec du code en secours, auto-hébergeable ou en SaaS¹.

Monorepo pnpm piloté par Turborepo : `pnpm-workspace.yaml`, `turbo.json` et un `package.json` racine `private: true` en version 2.39.0 ; tout le code applicatif vit sous `packages/`.

Quatre points d'entrée à connaître :

- `packages/cli/bin/n8n` — binaire visé par les scripts `start`, `webhook` et `worker` : contrôle la version de Node, charge `dist/config` puis le `CommandRegistry`.
- `packages/core` — moteur d'exécution des workflows.
- `packages/nodes-base` — les nœuds intégrés, c'est-à-dire l'essentiel de la surface fonctionnelle.
- `packages/frontend` — l'éditeur web.

Contraintes de plateforme fortes et récentes : Node ≥ 24 , pnpm $\geq 12.3.4$ imposé (`preinstall` bloque npm), TypeScript 6.0.2, Biome et Prettier côte à côte.

La racine est instrumentée comme un dépôt d'entreprise : `OWNERS`, `CODEOWNERS`, deux fichiers de licence, quatre configs d'agents IA (`AGENTS.md`, `CLAUDE.md`, `.claude/`, `.opencode`), trois configs de sécurité (`.aikido`, `.poutine.yml`, `security/`) et deux baselines gelées (`.boundaries-baseline.json`, `.code-health-baseline.json`).

Quelle est sa santé technique ?

À retenir : ~250 commits par semaine encaissés par une CI à merge queue avec check bloquant — la vélocité n'est pas payée en qualité.



Cadence rare : les 500 derniers changements relevés tiennent tous dans deux semaines, 251 puis 249, soit ~250 commits par semaine. Les barres à zéro sont la limite du relevé (500 commits), pas un arrêt : l'historique complet est dans le journal des commits du dépôt².

Dernier commit le jour même du relevé, dépôt non archivé, branche par défaut `master`.

Livraison en continu et sur canaux : les dix dernières versions relevées tiennent en quatre jours, avec des pointeurs mobiles `stable` et `beta`. Deux lignes vivent en parallèle — `n8n@2.39.2` et `n8n@1.123.79` le même jour : la 1.x est encore maintenue, ce qui est une bonne nouvelle pour qui déploie aujourd'hui.

Chaîne de build standard, sans surprise :

```
pnpm install    # installation
pnpm start      # démarrage
pnpm test       # tests (turbo run test)
```

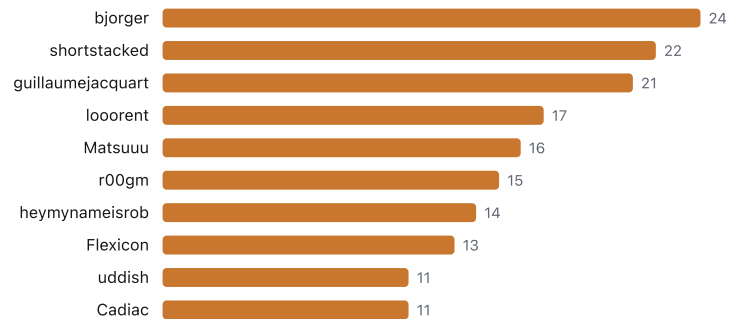
La CI est le vrai actif technique. `.github/workflows/ci-pull-requests.yml` tourne sur `pull_request` et `merge_group` : build, tests unitaires, typecheck, lint, `pack --dry-run`, e2e Playwright, tests DB SQLite + Postgres, e2e de performance, contrôles de sécurité, évaluations IA, et un job `required-checks` bloquant pour le merge³.

Autour : plus de cent workflows, dont une trentaine dédiés à la release (backport, patch PR, promotion de tag, publication npm) et une dizaine à la sécurité (SBOM, Trivy, Poutine, zizmor).

Tests répartis par package, pas de dossier racine : `vitest.workspace.ts`, six configs Vitest rien que pour `packages/cli` (unit, integration, migration, testcontainers), Playwright dans `packages/testing`, et du test de mutation via `@stryker-mutator/vitest-runner`. Le taux de couverture réel n'est pas relevé — Codecov est branché (`codecov.yml`), le chiffre est à lire sur le tableau de bord public du projet⁴.

Qui porte le projet et quel est le bus factor ?

À retenir : bus factor 16 et aucun contributeur au-dessus de 5 % du volume — le risque humain est faible, mais la roadmap se décide dans Linear, hors de votre vue.



Bus factor 16 : il faut réunir seize personnes pour couvrir la moitié des changements relevés. Au-delà de 10, le risque de dépendance individuelle est résiduel.

Aucune concentration : le plus actif pèse 24 changements sur 500, soit moins de 5 %, et les dix premiers cumulent 166 sur 500, soit un tiers.

Le dépôt appartient à une organisation, créée en juin 2019 — sept ans d'existence, pas une expérimentation.

Gouvernance des revues outillée : `CODEOWNERS` et un fichier `OWNERS` par zone, avec `ci-owners-required-reviews.yml`, `ci-owners-assign-reviewers.yml` et `ci-owners-validation.yml` pour l'appliquer en CI.

Point à connaître avant de compter sur GitHub comme source de vérité : 297 des issues relevées portent le label `status:in-linear` et 280 `status:team-assigned` — le pilotage réel se fait dans Linear, GitHub est la façade publique.

Pas de page de financement communautaire : le projet est financé par son éditeur et ses licences entreprise, pas par des dons.

Quelles dépendances et quelle dette ?

À retenir : zéro dépendance runtime à la racine, un verrou pnpm centralisé et deux baselines de dette — l'inventaire précis passe par les SBOM de la CI, pas par le package.json .

Le manifeste racine ne déclare aucune dépendance runtime : 28 entrées, toutes en devDependencies (turbo 2.9.15, typescript 6.0.2, eslint, prettier, biome, lefthook, zx...). Les dépendances de production sont déclarées package par package, avec catalogue et verrou centralisés (pnpm-workspace.yaml , pnpm-lock.yaml). Conséquence directe : **l'inventaire réel des dépendances runtime n'est pas relevé** ici ; il se lit dans les SBOM générés en CI (sbom-generation-callable.yml , test-sbom-nightly.yml).

Signaux de dette, dans l'ordre de ce qu'ils coûtent :

SIGNAL	CE QU'ON VOIT	LECTURE
patches/ à la racine	patches pnpm appliqués à des dépendances tierces	dette externe assumée : chaque montée de version tierce peut casser un patch
Deux baselines gelées	.boundaries-baseline.json , .code-health-baseline.json	la dette est mesurée et plafonnée, pas ignorée — mais elle existe et est datée
Stock ouvert	375 issues ouvertes, 756 PR ouvertes	l'équipe absorbe le flux sans faire baisser le stock
Toolchain de pointe	Node ≥ 24, pnpm ≥ 12.3.4, TypeScript 6.0.2	à aligner sur vos environnements avant tout build interne

La recherche de marqueurs TODO remonte 564 fichiers, mais elle est textuelle : une partie des résultats sont des fonctionnalités « to-do » du produit (nœud Microsoft To Do, outils write-todos des agents), pas de la dette. Chiffre inexploitable tel quel.

Quels risques ?

À retenir : le seul risque qui remonte au comité est juridique — `.ee.` et branches hors `master` non licenciés, usage strictement interne sans licence entreprise.

- license** Licence non standard « Sustainable Use License » : lire LICENSE.md avant tout usage.
- deps** package.json racine : 28 devDependencies, aucun bloc dependencies ; les dépendances runtime sont déclarées dans chaque package du monorepo (pnpm-workspace.yaml, pnpm-lock.yaml)
- ci** ci-pull-requests.yml tourne sur pull_request et merge_group : build, tests unitaires, typecheck, lint, packaging, e2e ; nombreux autres workflows (release, sécurité, nightly) dans .github/w...

Trois risques relevés, un seul bloquant, tous avec une parade connue.

RISQUE	NIVEAU	PARADE
Licence non standard « Sustainable Use License »	MOYEN	Usage interne : autorisé sans contrepartie. Revente, SaaS ou intégration à une offre payante : licence entreprise à négocier avant tout engagement
Dépendances runtime éclatées par package	FAIBLE	Verrou et catalogue pnpm centralisés, SBOM générés en CI : brancher <code>sbom-generation-callable.yml</code> sur votre chaîne d'audit
Dépendance à la CI de l'éditeur	FAIBLE	Build, tests, typecheck, lint, packaging et e2e sont dans le dépôt et rejouables : rien de la validation n'est un service privé

Trois clauses de `LICENSE.md` à faire lire au juridique, elles vont plus loin que le résumé du README⁵ :

- L'usage est limité à vos **propres besoins internes**, ou non commercial ; la redistribution n'est permise que gratuitement et à but non commercial.
- Tout fichier contenant `.ee.` dans son nom ou `.ee` dans son chemin **n'est pas** couvert : il exige une licence entreprise (`LICENSE_EE.md`).
- Le contenu des branches autres que `master` **n'est pas licencié** — ce qui a un effet direct sur toute stratégie de fork ou de branche longue.

Versant sécurité correctement outillé : politique de divulgation publiée, scanners Aikido, Poutine, Trivy et zizmor en CI, chaîne `sec-*` de publication de correctifs. Non relevé en revanche : les CVE et avis publiés, à lire directement chez l'éditeur⁶.

Adopter, contribuer ou forker ?

À retenir : adopter et suivre l'amont ; contribuer si vous acceptez le CLA ; ne pas forker — la licence l'interdit sur les branches et la cadence le rend intenable.

Ce qui se construit, lu dans les PR ouvertes : un plan de données séparé pour les exécutions, une pagination par curseur de bout en bout, un service de publication des workflows activé par défaut, et des applications capables de lire et écrire des tables de données — n8n dépasse l'automatisation d'arrière-plan.

Deux signaux à ne pas manquer : une PR de comparaison « 3.x » est ouverte, et une PR marquée rupture supprime le nœud LangChain Code. Une majeure se prépare et les ruptures sont assumées : verrouillez votre version et suivez les branches `release-candidate/*`.

Côté utilisateurs, les demandes les plus discutées convergent sur un même point de friction : faire fonctionner les agents et les outils MCP de façon fiable en production, y compris en mode queue. C'est aussi là que se situe le risque fonctionnel si votre cas d'usage est agentique.

Contribuer est possible mais encadré : CLA obligatoire vérifié par `ci-cla-check.yml`, revue imposée par les propriétaires de zone, règles explicites pour les PR communautaires et fermeture automatique des PR inactives (`community-pr-close-stale.yml`)⁷. Comptez le coût du CLA en amont, pas au moment du merge.

Forker est le mauvais calcul malgré 60 629 forks existants : la licence ne couvre pas les branches hors `master`, les fichiers `.ee.` en sont exclus, et un rythme de ~250 commits par semaine rend tout fork divergent irrattrapable en quelques mois.

Décision

- **Adopter, en auto-hébergé, sur une version épinglée.** Ingénierie mature (CI à merge queue avec check bloquant, tests DB et e2e, scanners de sécurité), cadence de ~250 commits par semaine, bus factor 16 : aucun signal technique ne s'y oppose.
- **Condition bloquante avant tout engagement : validation juridique.** Usage interne uniquement sans licence entreprise ; fichiers `.ee.` exclus ; branches hors `master` non licenciées. Si le projet est de revendre ou d'embarquer n8n dans une offre payante, la décision est commerciale, pas technique.
- **Épingler la version et surveiller la 3.x.** Deux lignes maintenues en parallèle (1.123.79 et 2.39.2), une PR de comparaison 3.x ouverte et une suppression de nœud marquée rupture : pas de `latest` en production.
- **Ne pas forker, contribuer en amont si besoin** — CLA à signer, revue par propriétaires de zone ; un fork décroche en quelques mois au rythme actuel.
- **Deux angles morts à instruire vous-même** : l'inventaire des dépendances runtime (par les SBOM de la CI, pas par le manifeste racine) et les avis de sécurité publiés, non relevés ici⁶.

RÉFÉRENCES PUBLIQUES

Sources

1. <https://github.com/n8n-io/n8n>
2. <https://github.com/n8n-io/n8n/commits/master>
3. <https://github.com/n8n-io/n8n/blob/master/.github/workflows/ci-pull-requests.yml>
4. <https://app.codecov.io/gh/n8n-io/n8n>
5. <https://github.com/n8n-io/n8n/blob/master/LICENSE.md>
6. <https://github.com/n8n-io/n8n/security/advisories>
7. <https://github.com/n8n-io/n8n/blob/master/CONTRIBUTING.md>